Lecture 3 Notes

Quantitative Data Analysis

Introduction

- Today we will start to look at our data
- We will first consider how to use some simple statistical measures to describe our data
- As we do this, we will also learn a bit more about R
- We will begin by looking at our dependent variable

Why bother with descriptive statistics

- Our first round of descriptive statistics helps us spot errors, outliers and other anomalies in our data
- Do the values make sense? Are they feasible values? Is there anything we can compare them to?
- Descriptive statistics can help us understand our data and the patterns in it
- When writing up quantitative work, you need to provide a description of your data so readers can understand it
- Sometimes describing the data is so interesting that it will be the most valuable part of the work

Measures of central tendency

- One key question we might ask is what is a typical value of our dependent variable
- There are different ways of answering this question
- In statistics, we refer to these as *measures of central tendency*
- One of the most well known is the *arithmetic mean* which we often refer to simply as the average
- It is derived by adding up all the values in a sequence and dividing by the number of values
- We can do this manually or using the mean function
- Let's start with calculating the mean of 1, 9, 8, 2, 7, 3, and 2

Calculating the mean

- To calculate the sample mean of X we have

$$X = \frac{\sum_{i=1}^{n} x_i}{n}$$

- x_i is the ith observation of x
- n is the sample size

Calculating the mean

```
(1 + 9 + 8 + 2 + 7 + 3 + 2) / 7
```

```
## [1] 4.571429
```

- Or we can do

```
mean(c(1,9,8,2,7,3,2))
```

```
## [1] 4.571429
```

Mean hourly wage

- Let's have a look at the mean hourly wage in the sample
- There are a few ways to do this
- We will wrap the summarise function around the mean function

```
summarise(lfs,mean(HOURPAY))
```

```
## # A tibble: 1 × 1
##   `mean(HOURPAY)`
##             <dbl>
## 1            20.8
```

- The mean hourly wage is therefore £20.84

The distribution

- The mean is one representative value we could calculate
- However we know there will be substantial variation
- The concept of the *distribution* is very important in statistics
- One way to begin is to look at the maximum and minimum values
- We can use the same approach but also use the *min* and *max* functions

Maximum and minimum

```
summarise(lfs,mean(HOURPAY),
              min(HOURPAY),
              max(HOURPAY))
```

```
## # A tibble: 1 × 3
##   `mean(HOURPAY)` `min(HOURPAY)` `max(HOURPAY)`
##             <dbl>          <dbl>          <dbl>
## 1            20.8            6.4           98.8
```

- We can add labels to the output if we want

Maximum and minimum

```
summarise(lfs,mean_pay = mean(HOURPAY),
              minimum_pay = min(HOURPAY),
              maximum_pay = max(HOURPAY))
```

```
## # A tibble: 1 × 3
##   mean_pay minimum_pay maximum_pay
##      <dbl>       <dbl>       <dbl>
## 1     20.8         6.4        98.8
```

Variance

- We can measure more about the distribution using the *variance*
- As it sounds, this is about measuring how much variation there is in the data
- We capture this by looking at how far away each point is from the mean
- We could think about measuring the distance from the mean by calculating $\Sigma(x_i - x)$
- This would cause the positive and negative distances to cancel out
- We can get around this by squaring the difference i.e., $\Sigma(x_i - x)^2$

Variance

- We can put this together to get the expression for the sample variance

$$S^2 = \frac{\Sigma(x_i - x)^2}{n - 1}$$

- Squaring it has made it harder to interpret

- We can take the square root to reverse it
- This gives us the *standard deviation*, which is measured in the same units as our variable

Calculating the variance and standard deviation

```
summarise(lfs,
          wage_var = var(HOURPAY),
          wage_sd = sd(HOURPAY))
```

```
## # A tibble: 1 × 2
##   wage_var wage_sd
##      <dbl>   <dbl>
## 1     133.    11.5
```

Quantiles

- Another useful way to investigate the distribution of a variable is to use *quantiles*
- We can define how many quantiles we want although we do this in an indirect way
- The method calculates which values break the data into equal groups
- We use different names depending on our selection of the number of groups
- Commonly used quantiles are:
    - Quartiles for 4 groups
    - Quintiles for 5 groups
    - Deciles for 10 groups
- We specify *cut points* as proportions/probabilities, and we can add labels to the output

Quantiles

```
reframe(lfs, quartiles = quantile(HOURPAY, probs = c(0.25, 0.5, 0.75)),
             q = c("First quartile","Second Quartile","Third Quartile"))
```

```
## # A tibble: 3 × 2
##   quartiles q
##       <dbl> <chr>
## 1      12.9 First quartile
## 2      17.3 Second Quartile
## 3      25.1 Third Quartile
```

Interpreting Quartiles

- 25% of people earn less than £12.86
- 50% of people earn less than £17.3
- 75% of people earn less than £25.09.
- If we want we can get the quantiles to show the maximum and minimum values

Quantiles

```
reframe(lfs, quartiles = quantile(HOURPAY, probs = c(0,0.25, 0.5, 0.75,
1)),
              q = c("Minimum","First quartile","Second Quartile",
                  "Third Quartile","Maximum"))
```

```
## # A tibble: 5 × 2
##   quartiles q
##       <dbl> <chr>
## 1       6.4 Minimum
## 2      12.9 First quartile
## 3      17.3 Second Quartile
## 4      25.1 Third Quartile
## 5      98.8 Maximum
```

The median

- The second quartile is so special, that it has a name
- It is known as the *median* and is the value that splits the data in half
- If we arrange the data from the smallest to the largest value, then it is the value in the middle
- We can calculate it using the *median* function
- It is less sensitive to outliers than the mean
- For instance, the mean of 3, 5, 9, 10, 12 is 7.8 and the median is 9
- The mean of 3, 5, 9, 10, 1000 is 205.4 and the median is 9

Median pay compared to mean

```
summarise(lfs,mean_pay = mean(HOURPAY),
              median_pay = median(HOURPAY))
```

```
## # A tibble: 1 × 2
##   mean_pay median_pay
##      <dbl>      <dbl>
## 1     20.8       17.3
```

- We can see that the median is considerably lower than the mean
- It seems that some individuals on very high pay are skewing the measure

The summary function

- We can pull these measures out quickly using Base R's summary function
- If we call summary on a data frame, we get a summary for all variables
- We can also call it on a specific variable
- I'm going to cheat here and use a bit of Base R's coding style

```
summary(lfs$HOURPAY)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    6.40   12.87   17.30   20.84   25.09   98.82
```

- We name the data frame and then use the dollar operator to name a variable within the data frame

Variable types

- Hourly pay is a continuous ratio variable
- This allows us to calculate the mean, median, and quantiles
- Other measurement levels behave differently
- Let's consider an ordinal variable
- A good example in our data is the EMPLEN variable, which describes how long someone has been in work
- We can start by making a *frequency table*, also referred to as a *contingency table*

Frequency table

```
count(lfs, EMPLEN)
```

```
## # A tibble: 8 × 2
##   EMPLEN                       n
##   <fct>                    <int>
## 1 Less than 3 months         196
## 2 Three months, less than 6   268
## 3 6 months, less than 12      425
## 4 1 year, less than 2         900
## 5 2 years, less than 5       1777
## 6 5 years, less than 10      1316
## 7 10 years, less than 20     1417
## 8 20 years or more           1048
```

Central tendancy of an ordinal variable

- We can't calculate the average for a person
- We could calculate the median by looking at which category the middle person is in
- We run into a bit of a problem when we have an even number of people
- A more straightforward measure is the *mode* which tells us what the most common category is
- The mode can also be calculated for categorical data since it does not depend on ordering
- A common approach is to present a frequency table, often with percentages

Adding percentages to a table

- There are various ways to add percentages to a table
- We'll begin using the same approach as we used
- Before going further, we are going to learn a very useful R trick
- Often in our analysis we will want to perform several operations in a row
- Some programming languages have a *pipe* which allows us to connect different functions together
- The pipe takes data in one end, lets it flow through several functions, and the result flows out the other end
- We will begin with a simple example

Base R's pipe (an aside)

- The pipe from Tidyverse proved so popular that in Version 4, a native pipe was added to R
- It is written as |>
- It behaved a little differently to the Tidyverse pipe
- We will not use it, but you might see it appear in other sources

Calculating a frequency table

- For our last frequency table, we did count(lfs, EMPLEN)
- An identical result can be achieved by writing

```
lfs %>% count(EMPLEN)
```

```
## # A tibble: 8 × 2
##   EMPLEN                        n
##   <fct>                     <int>
## 1 Less than 3 months          196
## 2 Three months, less than 6   268
## 3 6 months, less than 12      425
## 4 1 year, less than 2         900
## 5 2 years, less than 5       1777
## 6 5 years, less than 10      1316
## 7 10 years, less than 20     1417
## 8 20 years or more           1048
```

What is going on?

- We start by writing the name of the data frame
- This is then piped into the count function
- Whatever has been passed through the pipe becomes the first argument of the following function
- Tidyverse functions typically take a data frame as the first argument and produce a data frame at the end
- We can use this approach to chain together several operations

Adding percentages to our table

```
lfs %>% count(EMPLEN) %>% mutate(percent = n / sum(n)*100)
```

```
## # A tibble: 8 × 3
##   EMPLEN                       n percent
##   <fct>                    <int>   <dbl>
## 1 Less than 3 months         196    2.67
## 2 Three months, less than 6  268    3.65
## 3 6 months, less than 12     425    5.78
## 4 1 year, less than 2        900   12.2
## 5 2 years, less than 5      1777   24.2
## 6 5 years, less than 10     1316   17.9
## 7 10 years, less than 20    1417   19.3
## 8 20 years or more          1048   14.3
```

What's going on

1. We name the data frame we want to use
2. The data frame is passed to the count function. It counts how many people are in each category of the EMPLEN variable and generates a table. The table contains a list of the levels of the EMPLEN factor and a variable called n which counts the people in each category
3. This new table is passed to mutate, which calculates a new variable called percent

How many men and women in our data?

```
lfs %>% count(SEX) %>% mutate(percent = n / sum(n)*100)
```

```
## # A tibble: 2 × 3
##   SEX        n percent
##   <fct>  <int>   <dbl>
## 1 Male    3410    46.4
## 2 Female  3937    53.6
```

- We have slightly more women than men in the sample

Where do people live?

```
lfs %>% count(GOR9D) %>% mutate(percent = n / sum(n)*100)
```

```
## # A tibble: 12 × 3
##    GOR9D                        n percent
##    <fct>                    <int>   <dbl>
##  1 London                     581    7.91
##  2 Yorkshire and The Humber   761   10.4
##  3 South West                 747   10.2
##  4 West Midlands              555    7.55
##  5 South East                1121   15.3
##  6 East of England            602    8.19
##  7 East Midlands              654    8.90
##  8 North East                 290    3.95
##  9 North West                 664    9.04
## 10 Wales                      282    3.84
## 11 Northern Ireland           473    6.44
## 12 Scotland                   617    8.40
```

How many on zero hours contracts?

```
lfs %>% count(FLEX22W6) %>% mutate(percent = n / sum(n)*100)
```

```
## # A tibble: 2 × 3
##   FLEX22W6     n percent
##   <fct>    <int>   <dbl>
## 1 Yes        152    2.07
## 2 No        7195   97.9
```

Full-time / part-time split

```
lfs %>% count(FTPT) %>% mutate(percent = n / sum(n)*100)
```

```
## # A tibble: 2 × 3
##   FTPT          n percent
##   <fct>     <int>   <dbl>
## 1 Full-time  5762    78.4
## 2 Part-time  1585    21.6
```

Education split

```
lfs %>% count(HIQUL22D) %>% mutate(percent = n / sum(n)*100)
```

```
## # A tibble: 6 × 3
##   HIQUL22D                    n percent
##   <fct>                   <int>   <dbl>
## 1 Degree or equivalent     3535   48.1
## 2 Higher education          565    7.69
## 3 A level or equivalent    1604   21.8
## 4 GCSE A*-C or equivalent  1102   15.0
## 5 Other qualitfication      331    4.51
## 6 No Qualification          210    2.86
```

People by industry

```
lfs %>% count(INDE07M) %>% mutate(percent = n / sum(n)*100)
```

```
## # A tibble: 9 × 3
##   INDE07M                                n percent
##   <fct>                              <int>   <dbl>
## 1 Agriculture, forestry and fishing     33   0.449
## 2 Energy and water                     155   2.11
## 3 Manufacturing                        636   8.66
## 4 Construction                         332   4.52
## 5 Distribution, hotels and restaurants 1026  14.0
## 6 Transport and communication          693   9.43
## 7 Banking and finance                 1311  17.8
## 8 Public admin, education and health  2858  38.9
## 9 Other services                       303   4.12
```

Number in permanent and temporary positions

```
lfs %>% count(JOBTYP) %>% mutate(percent = n / sum(n)*100)
```

```
## # A tibble: 2 × 3
##   JOBTYP                      n percent
##   <fct>                   <int>   <dbl>
## 1 Permanent                7106   96.7
## 2 Not permanent in some way  241    3.28
```

Number of employees at work

```
lfs %>% count(MPNR02) %>% mutate(percent = n / sum(n)*100)
```

```
## # A tibble: 9 × 3
##   MPNR02                                n percent
##   <fct>                             <int>   <dbl>
## 1 1–10                               1173   16.0
## 2 11–19                               560    7.62
## 3 20–24                               279    3.80
## 4 Don't know but under 25             151    2.06
## 5 25–49                               875   11.9
## 6 50–249                             1681   22.9
## 7 250–499                             477    6.49
## 8 Don't know but between 50 and 499   312    4.25
## 9 500 or more                        1839   25.0
```

People by marital status

```
lfs %>% count(MARSTA) %>% mutate(percent = n / sum(n)*100)
```

```
## # A tibble: 6 × 3
##   MARSTA                                          n percent
##   <fct>                                       <int>   <dbl>
## 1 Single, never married                        2699   36.7
## 2 Married, living with spouse                  3785   51.5
## 3 Married separated from spouse                 166    2.26
## 4 Divorced                                      546    7.43
## 5 Widowed                                        74    1.01
## 6 Currently or previously in civil partnership   77    1.05
```

People by seniority

```
lfs %>% count(NSECMJ20) %>% mutate(percent = n / sum(n)*100)
```

```
## # A tibble: 7 × 3
##   NSECMJ20                              n percent
##   <fct>                             <int>   <dbl>
## 1 Higher managerial and professional  1836   25.0
## 2 Lower managerial and professional   2426   33.0
## 3 Intermediate occupations            1112   15.1
## 4 Lower supervisory and technical      423    5.76
## 5 Semi-routine occupations             728    9.91
## 6 Routine occupations                  662    9.01
## 7 Never worked                         160    2.18
```

Summary of basic usual hours

```
summary(lfs$BUSHR)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    7.00   32.00   38.00   34.76   40.00   90.00
```

Summary of basic usual hours

- We can also use the summary function in a Tidyverse way

```
lfs %>% pull(BUSHR) %>% summary()
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    7.00   32.00   38.00   34.76   40.00   90.00
```

Summary statistics by group

- So far we have looked at univariate measures i.e., one variable
- Things get more interesting when we start to summarise things by group
- We can do this easily
- We will make use of the *group_by* function
- This function changes the way the summarise function behaves
- Let's calculate our main measure of interest, the difference in average hourly wage between men and women

Average and median wage for men and women

```
lfs %>% group_by(SEX) %>% summarise(mean_wage = mean(HOURPAY),
                                    median_wage = median(HOURPAY))
```

```
## # A tibble: 2 × 3
##   SEX    mean_wage median_wage
##   <fct>      <dbl>       <dbl>
## 1 Male        22.5        18.5
## 2 Female      19.4        16.4
```

What next

- We can see quite a large difference in average wages

- One explanation of the difference is that men and women differ in characteristics which determine wages
- For instance, suppose they have different levels of education on average
- We can examine this using a *cross-tabulation*
- A handy way to do this is to use Base R's *table* function
- The table function is from Base R, so it behaves a little differently to the functions we have met so far
- It doesn't take a data frame as its first argument, and we need to feed it the variables directly
- We can do this using the dollar notation we met earlier

Sex and education

```
table(lfs$HIQUL22D, lfs$SEX)
```

```
##
##                           Male Female
##   Degree or equivalent    1511   2024
##   Higher education         269    296
##   A level or equivalent    816    788
##   GCSE A*–C or equivalent  531    571
##   Other qualitfication     177    154
##   No Qualification         106    104
```

Proportions

- The absolute counts are hard to interpret
- It would be more informative to look at proportions / percentages
- We could have three types of percentages
    - Cell percentages - value as a percentage of total in table
    - Row percentages - value as a percentage of total in that row
    - Column percentages - value as a percentage of total in that column
- We can use the *prop.table* function to do this
- Its *margin* argument controls what we get
- We can use the round function to control the number of decimal places we get

Sex and education - cell proportions

```
table(lfs$HIQUL22D, lfs$SEX) %>% prop.table() %>% round(2)
```

```
##
##                            Male Female
##   Degree or equivalent    0.21   0.28
##   Higher education        0.04   0.04
##   A level or equivalent   0.11   0.11
##   GCSE A*-C or equivalent 0.07   0.08
##   Other qualitfication    0.02   0.02
##   No Qualification        0.01   0.01
```

Sex and education - row proportions

```
table(lfs$HIQUL22D, lfs$SEX) %>% prop.table(1) %>% round(2)
```

```
##
##                            Male Female
##   Degree or equivalent    0.43   0.57
##   Higher education        0.48   0.52
##   A level or equivalent   0.51   0.49
##   GCSE A*-C or equivalent 0.48   0.52
##   Other qualitfication    0.53   0.47
##   No Qualification        0.50   0.50
```

Sex and education - column proportions

```
table(lfs$HIQUL22D, lfs$SEX) %>% prop.table(2) %>% round(2)
```

```
##
##                            Male Female
##   Degree or equivalent    0.44   0.51
##   Higher education        0.08   0.08
##   A level or equivalent   0.24   0.20
##   GCSE A*-C or equivalent 0.16   0.15
##   Other qualitfication    0.05   0.04
##   No Qualification        0.03   0.03
```

Sex and industry - column proportions

```
table(lfs$INDE07M, lfs$SEX) %>% prop.table(2) %>% round(2)
```

```
##
##                                       Male Female
##   Agriculture, forestry and fishing    0.01   0.00
##   Energy and water                     0.03   0.01
##   Manufacturing                        0.13   0.05
##   Construction                         0.07   0.02
##   Distribution, hotels and restaurants 0.15   0.13
##   Transport and communication          0.14   0.05
##   Banking and finance                  0.19   0.17
##   Public admin, education and health   0.24   0.52
##   Other services                       0.05   0.04
```

Some other useful functions

Inspecting your data

- The measures we have covered so far are the kind of thing you will present in your write-up
- There are some other useful functions for looking at your data
- We have already met *glimpse* which will present a brief summary of the variable types in your data
- We have also met *View* which will open up a spreadsheet of our data
- The *head* function will print the first five lines of the data frame to the console
- You can change the number of lines printed by setting the *n* argument

The head function

```
head(lfs)
```

```
## # A tibble: 6 × 14
##   SEX   HOURPAY   AGE GOR9D EMPLEN FLEX22W6 FTPT  HIQUL22D INDE07M JOB
TYP MPNR02
##   <fct>   <dbl> <dbl> <fct> <fct>  <fct>    <fct> <fct>    <fct>   <fc
t>  <fct>
## 1 Male     14.4    61 Sout… 6 mon… No       Part… GCSE A*… Other … Per
ma… 1-10
## 2 Male     11.6    60 Sout… Three… No       Full… GCSE A*… Public… Per
ma… 1-10
## 3 Fema…    10.4    63 Sout… 2 yea… No       Full… A level… Public… Per
ma… 1-10
## 4 Fema…    25.3    54 Sout… 1 yea… No       Full… Degree … Public… Not
p… 500 o…
## 5 Fema…    13.8    39 Sout… 2 yea… No       Full… Degree … Public… Per
ma… 1-10
## 6 Male     41.0    53 Sout… 10 ye… No       Full… Degree … Transp… Per
ma… 11-19
## # ℹ 3 more variables: MARSTA <fct>, NSECMJ20 <fct>, BUSHR <dbl>
```

The tail and arrange functions

- The tail function works much like the head function but prints out the last five rows
- This isn't much use if the order of the data is random
- We can sort the data using the *arrange* function
- We can supply the variable we want to use to do the sorting in ascending order
- A negative sign before the variable sorts in descending order
- Let's looks at the highest paid individuals

Arranging and inspecting the data

```
lfs %>% arrange(HOURPAY) %>% tail()
```

```
## # A tibble: 6 × 14
##   SEX   HOURPAY   AGE GOR9D EMPLEN FLEX22W6 FTPT  HIQUL22D INDE07M JOB
TYP MPNR02
##   <fct>   <dbl> <dbl> <fct> <fct>  <fct>    <fct> <fct>    <fct>   <fc
t>  <fct>
## 1 Male     94.0    39 Lond… 2 yea… No       Full… A level… Transp… Per
ma… 1–10
## 2 Fema…    95.4    45 Sout… 10 ye… No       Full… Degree … Constr… Per
ma… 50–249
## 3 Male     96.1    56 Lond… 1 yea… No       Part… Degree … Public… Per
ma… 1–10
## 4 Male     96.2    58 East… 20 ye… No       Full… A level… Transp… Per
ma… 250–4…
## 5 Male     96.2    45 Scot… 2 yea… No       Full… Degree … Public… Per
ma… 500 o…
## 6 Fema…    98.8    51 Sout… 6 mon… No       Full… Degree … Bankin… Per
ma… 500 o…
## # ℹ 3 more variables: MARSTA <fct>, NSECMJ20 <fct>, BUSHR <dbl>
```

Summary

- It's important to look at and summarise your data
- This will help you to understand it and will help your readers to understand it
- You may find interesting patterns and find ways to present them
- Next time we will visualise the data
- This extends what we have done today